



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

Data Structures & Algorithms in Python

Linked Lists – Tutorial Answers

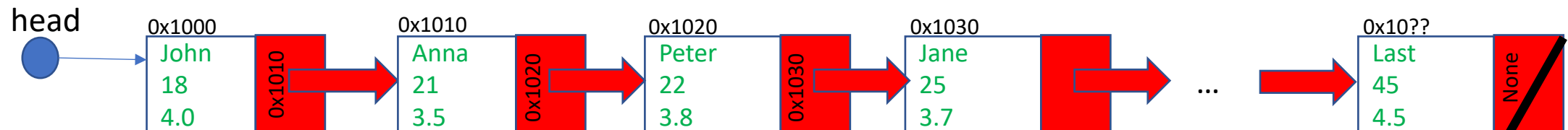
Dr. Owen Noel Newton Fernando

College of Engineering

School of Computer Science and Engineering

linked lists Recap

- A **LinkedList** consists of a connected sequence of **Link/Node** objects
- Each **node** contains a reference to a datum and a reference to the next **node** in the sequence.
 - The datum is just an object. It could be just an int or a string, or it could contain a key/data pair.
- The last node in the sequence refers to **None** as its next **node**
- The sequence of **nodes** is accessed via a “header” object that contains a reference to the first **node** in the sequence.



1. (moveEvenItemsToBackLL)

Write a Python function `move_even_items_to_back_ll()` that moves all the even integers to the back of the linked list.

The function prototype is given as follows:

```
def move_even_items_to_back(ll: LinkedList):
```

Q01 - Answer

Let's go through the step-by-step

1. Class Definitions

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

- **Node** represents a single element of the linked list.
- `data` holds the value, `next` points to the next node (or `None` if it's the last node).

Q01 - Answer

1. Class Definitions

```
class LinkedList:  
    def __init__(self):  
        self.head = None  
        self.size = 0
```

class LinkedList:

This starts the definition of a new class called LinkedList

Q01 - Answer

2. Functions

Finding a node by index

- If there's no head node, the list is empty.
- There's no point searching, so it raises an error.



```
def findNode(self, index):
```

```
    if self.head is None:  
        raise ValueError("List is empty")  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

Q01 - Answer

2. Functions

Finding a node by index

Checks if the index is **out of bounds**.

For example:

If size = 5, valid indexes are 0 to 4.

Negative indexes are also not allowed here.

If invalid → raise an error immediately.



```
def findNode(self, index):  
  
    if self.head is None:  
        raise ValueError("List is empty")  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

Q01 - Answer

2. Functions

Finding a node by index

```
def findNode(self, index):  
  
    if self.head is None:  
        raise ValueError("List is empty")  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

- Starts traversal from the first node in the list.



Q01 - Answer

2. Functions

- **Finding a node by index**

- Moves forward through the linked list index times.
- Each loop moves cur to the next node and decrements index.
- When index reaches 0, cur is pointing to the desired node.



```
def findNode(self, index):  
  
    if self.head is None:  
        raise ValueError("List is empty")  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

Q01 - Answer

2. Functions

- Finding a node by index

```
def findNode(self, index):  
  
    if self.head is None:  
        raise ValueError("List is empty")  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    cur = self.head  
    while index > 0:  
        cur = cur.next  
        index -= 1  
    return cur
```

- Returns the node object at the requested position.

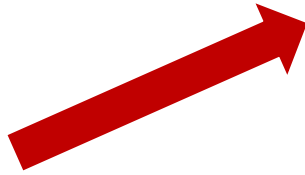


Q01 - Answer

2. Functions

- **Inserting a node**

- Validates that the insertion position is allowed.
- You can insert at self.size (end of list), but not beyond.
- Example: If size = 3, valid indexes are 0, 1, 2, 3.



```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")  
  
    new_node = Node(data)  
  
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True  
  
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- **Inserting a node**

- Creates a new node with the given data.
- Initially, new_node.next is None.



```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")  
  
    new_node = Node(data)  
  
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True  
  
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- **Inserting a node**

- Special case: inserting at the start.
- The new node's next is set to the current head.
- The head is updated to be the new node.
- Increases the list size by 1.
- Returns True to indicate success.



```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")
```

```
    new_node = Node(data)
```

```
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True
```

```
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- Inserting a node

- For any position other than 0, you first need to find the node before the insertion point.
- Uses your **findNode()** method to get it.
- Example: inserting at index 3 → you first find index 2.

```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")  
  
    new_node = Node(data)  
  
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True  
  
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- Inserting a node

- Links the new node into the list:
 - **new_node.next** points to what **prev_node** used to point to.
 - **prev_node.next** is updated to point to the new node.
- Increments size.
- Returns True if succeeds, else false.

```
def insertNode(self, data, index):  
    if index < 0 or index > self.size:  
        raise ValueError("Invalid position")
```

```
    new_node = Node(data)
```

```
    if index == 0:  
        new_node.next = self.head  
        self.head = new_node  
        self.size += 1  
        return True
```

```
    prev_node = self.findNode(index - 1)  
    if prev_node is not None:  
        new_node.next = prev_node.next  
        prev_node.next = new_node  
        self.size += 1  
        return True  
    return False
```



Q01 - Answer

2. Functions

- **Removing a node**

- Makes sure the index is within valid bounds ($0 \leq \text{index} < \text{size of the list}$).
- If it's not, a ValueError is raised.



```
def removeNode(self, index):  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    if self.head is None:  
        return False  
  
    if index == 0:  
        cur = self.head  
        self.head = cur.next  
        self.size -= 1  
        return True  
  
    pre = self.findNode(index - 1)  
    if pre is not None and pre.next is not None:  
        cur = pre.next  
        pre.next = cur.next  
        self.size -= 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- Removing a node

- If the list has no nodes, removal is impossible, so it returns False.



```
def removeNode(self, index):  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")  
  
    if self.head is None:  
        return False  
  
    if index == 0:  
        cur = self.head  
        self.head = cur.next  
        self.size -= 1  
        return True  
  
    pre = self.findNode(index - 1)  
    if pre is not None and pre.next is not None:  
        cur = pre.next  
        pre.next = cur.next  
        self.size -= 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- **Removing a node**

- If the first node (**index == 0**) is being removed:
 - Store the current head in cur.
 - Update the head to the second node (cur.next).
 - Reduce the size by 1.
 - Return True indicating success.



```
def removeNode(self, index):
    if index < 0 or index >= self.size:
        raise ValueError("Invalid position")

    if self.head is None:
        return False

    if index == 0:
        cur = self.head
        self.head = cur.next
        self.size -= 1
        return True

    pre = self.findNode(index - 1)
    if pre is not None and pre.next is not None:
        cur = pre.next
        pre.next = cur.next
        self.size -= 1
        return True
    return False
```

Q01 - Answer

2. Functions

• Removing a node

- First, find the previous node (**index - 1**) using `findNode()`.
- Make sure `pre` exists and that there's a node after it (**`pre.next`**).
- Store the node to be deleted in `cur`.
- Link the previous node directly to the node after the one being deleted (**`pre.next = cur.next`**), skipping `cur`.
- Reduce the size by 1.
- Return `True`.



```
def removeNode(self, index):  
    if index < 0 or index >= self.size:  
        raise ValueError("Invalid position")
```

```
    if self.head is None:  
        return False
```

```
    if index == 0:  
        cur = self.head  
        self.head = cur.next  
        self.size -= 1  
        return True
```

```
    pre = self.findNode(index - 1)  
    if pre is not None and pre.next is not None:  
        cur = pre.next  
        pre.next = cur.next  
        self.size -= 1  
        return True  
    return False
```

Q01 - Answer

2. Functions

- printing the entire linked list

- **cur** starts at the head of the list.



```
def printList(self):  
    cur = self.head  
    if cur is None:  
        print("Empty")  
        return  
  
    while cur is not None:  
        print(cur.data, end=" -> ")  
        cur = cur.next  
    print("None")
```

Q01 - Answer

2. Functions

- printing the entire linked list

- If the head is None, the list is empty.
- Print "Empty" and return immediately since there is nothing to print.



```
def printList(self):  
    cur = self.head  
    if cur is None:  
        print("Empty")  
        return  
  
    while cur is not None:  
        print(cur.data, end=" -> ")  
        cur = cur.next  
    print("None")
```

Q01 - Answer

2. Functions

- **printing the entire linked list**

- Loop through the list starting from the head.
- For each node, print its data followed by " -> " without moving to a new line (end=" -> ").
- Move cur to the next node.



```
def printList(self):  
    cur = self.head  
    if cur is None:  
        print("Empty")  
        return
```

```
while cur is not None:  
    print(cur.data, end=" -> ")  
    cur = cur.next  
print("None")
```

Q01 - Answer

2. Functions

- printing the entire linked list

- After the loop finishes (i.e., when cur becomes None), print "None" to indicate the end of the list.



```
def printList(self):  
    cur = self.head  
    if cur is None:  
        print("Empty")  
        return  
  
    while cur is not None:  
        print(cur.data, end=" -> ")  
        cur = cur.next  
    print("None")
```

2. Functions

- Removing all nodes

- **self.head = None**
 - In a linked list, **head** points to the first node.
 - Setting **self.head** to **None** removes the reference to the first node.
 - Since there's no reference to the first node, Python's garbage collector will eventually clean up all the nodes (because each node is only linked through **head**).
- **self.size = 0**
 - Updates the linked list's stored size counter to 0, indicating it's now empty.



```
def removeAllItems(self):  
    self.head = None  
    self.size = 0
```

Q01 - Answer

2. Functions

Now we will look at the `moveEvenItemsToBack()` Function

- If the list has fewer than 2 elements, there's nothing to rearrange, so the function returns immediately.



```
def moveEvenItemsToBack(ll):  
    if ll.size < 2:  
        return  
  
    cur = ll.head  
    index = 0  
    for _ in range(ll.size):  
        if cur.item % 2 == 0:  
            even_value = cur.item  
            cur = cur.next  
            ll.removeNode(index)  
            ll.insertNode(even_value, ll.size)  
        else:  
            cur = cur.next  
            index += 1
```

Q01 - Answer

2. Functions

Now we will look at the `moveEvenItemsToBack()` Function

- **cur** starts at the head of the linked list.
- **index** keeps track of the current node's position as we traverse the list.



```
def moveEvenItemsToBack(ll):  
    if ll.size < 2:  
        return  
  
    cur = ll.head  
    index = 0  
    for _ in range(ll.size):  
        if cur.item % 2 == 0:  
            even_value = cur.item  
            cur = cur.next  
            ll.removeNode(index)  
            ll.insertNode(even_value, ll.size)  
        else:  
            cur = cur.next  
            index += 1
```

Q01 - Answer

2. Functions

Now we will look at the `moveEvenItemsToBack()` Function

- Loop exactly `ll.size` times.
- This is important because we will be modifying the list while iterating.



```
def moveEvenItemsToBack(ll):  
    if ll.size < 2:  
        return  
  
    cur = ll.head  
    index = 0  
    for _ in range(ll.size):  
        if cur.item % 2 == 0:  
            even_value = cur.item  
            cur = cur.next  
            ll.removeNode(index)  
            ll.insertNode(even_value, ll.size)  
        else:  
            cur = cur.next  
            index += 1
```

Q01 - Answer

2. Functions

Now we will look at the moveEvenItemsToBack() Function

If the current node's value is even:

- Store the even value in **even_value**.
- Move **cur** forward to the next node (so we don't lose track).
- Remove the node at **index** (the current position).
- Insert the stored even value at the end of the list (**ll.size** is always the current list length, so insertion at **ll.size** appends to the end).
- Note: After removing, the list size decreases by 1, but inserting at the end keeps the size same.

```
def moveEvenItemsToBack(ll):  
    if ll.size < 2:  
        return  
  
    cur = ll.head  
    index = 0  
    for _ in range(ll.size):  
        if cur.item % 2 == 0:  
            even_value = cur.item  
            cur = cur.next  
            ll.removeNode(index)  
            ll.insertNode(even_value, ll.size)  
        else:  
            cur = cur.next  
            index += 1
```



Q01 - Answer

2. Functions

Now we will look at the `moveEvenItemsToBack()` Function

```
def moveEvenItemsToBack(ll):  
    if ll.size < 2:  
        return  
  
    cur = ll.head  
    index = 0  
    for _ in range(ll.size):  
        if cur.item % 2 == 0:  
            even_value = cur.item  
            cur = cur.next  
            ll.removeNode(index)  
            ll.insertNode(even_value, ll.size)  
        else:  
            cur = cur.next  
            index += 1
```

If the current node's value is odd:

- Just move **cur** forward.
- Increase **index** because the node stays in place.



Q01 - Answer

Testing our function

- It creates an empty linked list object `ll`.



```
if __name__ == "__main__":
```

```
    ll = LinkedList()
```

```
    while True:
```

```
        print("1: Insert an integer to the linked list:")
```

```
        print("2: Move all even integers to the back of the  
linked list:")
```

```
        print("0: Quit:")
```

```
        choice = int(input("Please input your choice (1/2/0): "))
```

#Continued...

Q01 - Answer

Testing our function

- Starts an infinite loop to repeatedly show a menu with three options:
 - Insert a new integer into the linked list
 - Move all even integers to the back of the list
 - Quit the program



```
if __name__ == "__main__":  
    ll = LinkedList()
```

```
    while True:  
        print("1: Insert an integer to the linked list:")  
        print("2: Move all even integers to the back of the  
linked list:")  
        print("0: Quit:")  
  
        choice = int(input("Please input your choice (1/2/0): "))
```

#Continued...

Q01 - Answer

Testing our function

```
if __name__ == "__main__":  
    ll = LinkedList()  
  
    while True:  
        print("1: Insert an integer to the linked list:")  
        print("2: Move all even integers to the back of the  
linked list:")  
        print("0: Quit:")
```

- Takes user input as a choice and converts it to an integer.



```
choice = int(input("Please input your choice (1/2/0): "))
```

#Continued...

Q01 - Answer

Testing our function

- If user chooses 1, it prompts for an integer to insert.
- Inserts the integer at the end of the linked list (**ll.size** as the index appends to the end).
- Prints the updated linked list.



```
if choice == 1:
    value = int(input("Input an integer that you
want to add to the linked list: "))
    ll.insertNode(value, ll.size)
    print("The resulting linked list is:")
    ll.printList()
elif choice == 2:
    moveEvenItemsToBack(ll)
    print("The resulting linked list after moving
even integers to the back is:")
    ll.printList()
elif choice == 0:
    ll.removeAllItems()
    break
else:
    print("Choice unknown;")
```

Q01 - Answer

Testing our function

- Calls the function **moveEvenItemsToBack(ll)** which rearranges the linked list so that all even numbers are moved to the back.
 - Prints the resulting linked list after rearrangement.
- 

```
if choice == 1:
    value = int(input("Input an integer that you
want to add to the linked list: "))
    ll.insertNode(value, ll.size)
    print("The resulting linked list is:")
    ll.printList()
elif choice == 2:
    moveEvenItemsToBack(ll)
    print("The resulting linked list after moving
even integers to the back is:")
    ll.printList()
elif choice == 0:
    ll.removeAllItems()
    break
else:
    print("Choice unknown;")
```

Q01 - Answer

Testing our function

- Calls `ll.removeAllItems()` to clear the linked list.
- Breaks the infinite loop, ending the program.



```
if choice == 1:
    value = int(input("Input an integer that you
want to add to the linked list: "))
    ll.insertNode(value, ll.size)
    print("The resulting linked list is:")
    ll.printList()
elif choice == 2:
    moveEvenItemsToBack(ll)
    print("The resulting linked list after moving
even integers to the back is:")
    ll.printList()
elif choice == 0:
    ll.removeAllItems()
    break
else:
    print("Choice unknown;")
```

Q01 - Answer

Testing our function

- If the input is not 0, 1, or 2, prints an error message and loops again.



```
if choice == 1:
    value = int(input("Input an integer that you
want to add to the linked list: "))
    ll.insertNode(value, ll.size)
    print("The resulting linked list is:")
    ll.printList()
elif choice == 2:
    moveEvenItemsToBack(ll)
    print("The resulting linked list after moving
even integers to the back is:")
    ll.printList()
elif choice == 0:
    ll.removeAllItems()
    break
else:
    print("Choice unknown;")
```

2. (moveMaxToFront)

Write a Python function `move_max_to_front()` that traverses a linked list of integers at most once, then moves the node with the largest stored value to the front of the list.

The function prototype is given as follows:

`def move_max_to_front(ll: LinkedList):`

Q02 - Answer

Now we will go through the new moveMaxToFront(ll) Function

- If the list is empty (**ll.head** is **None**) or has only one element (**ll.head.next** is **None**), there's no need to move anything.
- Returns **-1** to indicate no operation was performed.



```
def moveMaxToFront(ll):  
    if ll.head is None or ll.head.next is None:  
        return -1  
  
    pre_max = None  
    max_node = ll.head  
    cur = ll.head  
  
    while cur.next is not None:  
        if cur.next.item > max_node.item:  
            pre_max = cur  
            max_node = cur.next  
            cur = cur.next  
  
    if pre_max is None:  
        return 0
```

#Continued...

Q02 - Answer

Now we will go through the new moveMaxToFront(ll) Function

- **pre_max** → Will store the node before the maximum node (so we can relink it when removing max node from its original position).
- **max_node** → Stores the node that currently holds the maximum value found so far.
- Initially, assume the first node is the max.
- **cur** → A pointer to traverse the list.



```
def moveMaxToFront(ll):  
    if ll.head is None or ll.head.next is None:  
        return -1  
  
    pre_max = None  
    max_node = ll.head  
    cur = ll.head  
  
    while cur.next is not None:  
        if cur.next.item > max_node.item:  
            pre_max = cur  
            max_node = cur.next  
            cur = cur.next  
  
    if pre_max is None:  
        return 0
```

#Continued...

Q02 - Answer

Now we will go through the new moveMaxToFront(ll) Function

- Traverses the linked list one node at a time.

Key trick:

- Instead of comparing **cur.item** directly, it compares **cur.next.item**.
- This way, if a new maximum is found, we already have the pointer to its previous node (**cur**), which is needed for re-linking.
- When a bigger value is found:
- Update **pre_max** to point to the node before the max node.
- Update **max_node** to point to this new maximum node.



```
def moveMaxToFront(ll):  
    if ll.head is None or ll.head.next is None:  
        return -1  
  
    pre_max = None  
    max_node = ll.head  
    cur = ll.head  
  
    while cur.next is not None:  
        if cur.next.item > max_node.item:  
            pre_max = cur  
            max_node = cur.next  
            cur = cur.next  
  
    if pre_max is None:  
        return 0
```

#Continued...

Q02 - Answer

Now we will go through the new moveMaxToFront(ll) Function

```
def moveMaxToFront(ll):  
    if ll.head is None or ll.head.next is None:  
        return -1  
  
    pre_max = None  
    max_node = ll.head  
    cur = ll.head  
  
    while cur.next is not None:  
        if cur.next.item > max_node.item:  
            pre_max = cur  
            max_node = cur.next  
            cur = cur.next
```

- If **pre_max** is still **None**, it means the largest value was already at the **front**.
- Nothing to move — **return 0**.



```
if pre_max is None:  
    return 0
```

#Continued...

Q02 - Answer

Now we will go through the new moveMaxToFront(ll) Function

- Remove **max_node** from its current spot: **pre_max.next = max_node.next**
- Insert **max_node** at the front:
 1. Point **max_node.next** to the old head.
 2. Update **ll.head** to be **max_node**.



```
pre_max.next = max_node.next  
max_node.next = ll.head  
ll.head = max_node  
  
return 0
```

Q02 - Answer

Now we will go through the new moveMaxToFront(ll) Function

- Returns 0 to indicate the operation was successful.



```
pre_max.next = max_node.next  
max_node.next = ll.head  
ll.head = max_node
```

```
return 0
```

3. (removeDuplicatesSortedLL)

Write a Python function `remove_duplicates_sorted_ll()` that removes all duplicate values from a sorted linked list. You may assume that the list is already in ascending sorted order.

The function prototype is given below:

`def remove_duplicates_sorted_ll(ll: LinkedList):`

Q03 - Answer

Now we will go through the `remove_duplicates_sorted_ll(ll)` Function

- `current` is a pointer to the first node in the linked list.



```
def remove_duplicates_sorted_ll(ll):  
    current = ll.head  
    if current is None:  
        return  
  
    while current.next is not None:  
        if current.item == current.next.item:  
            temp = current.next.next  
            current.next = temp  
            ll.size -= 1  
        else:  
            current = current.next
```

Q03 - Answer

Now we will go through the `remove_duplicates_sorted_ll(ll)` Function

- If `ll.head` is `None`, the list is **empty** → nothing to remove → exit function.



```
def remove_duplicates_sorted_ll(ll):  
    current = ll.head  
    if current is None:  
        return  
  
    while current.next is not None:  
        if current.item == current.next.item:  
            temp = current.next.next  
            current.next = temp  
            ll.size -= 1  
        else:  
            current = current.next
```

Q03 - Answer

Now we will go through the `remove_duplicates_sorted_ll(ll)` Function

- Loops until we reach the last node (`current.next` will be **None** at the last node).
- This means we're always comparing the **current node** with its next node.



```
def remove_duplicates_sorted_ll(ll):  
    current = ll.head  
    if current is None:  
        return  
  
    while current.next is not None:  
        if current.item == current.next.item:  
            temp = current.next.next  
            current.next = temp  
            ll.size -= 1  
        else:  
            current = current.next
```

Q03 - Answer

Now we will go through the `remove_duplicates_sorted_ll(ll)` Function

- Since the list is sorted, duplicates will always appear next to each other.
- If the **current** node's value is equal to the **next** node's value → we found a duplicate.



```
def remove_duplicates_sorted_ll(ll):  
    current = ll.head  
    if current is None:  
        return  
  
    while current.next is not None:  
        if current.item == current.next.item:  
            temp = current.next.next  
            current.next = temp  
            ll.size -= 1  
        else:  
            current = current.next
```

Q03 - Answer

Now we will go through the `remove_duplicates_sorted_ll(ll)` Function

Removing duplicate

- We “skip” over the duplicate node by making **current.next** point to the node after the duplicate.
- This removes the duplicate from the linked list.
- We reduce **ll.size** by 1 because we deleted a node.
- Important: **current** does NOT move forward in this case — we recheck the new **current.next** in case there are multiple duplicates in a row.



```
def remove_duplicates_sorted_ll(ll):  
    current = ll.head  
    if current is None:  
        return  
  
    while current.next is not None:  
        if current.item == current.next.item:  
            temp = current.next.next  
            current.next = temp  
            ll.size -= 1  
        else:  
            current = current.next
```

Q03 - Answer

Now we will go through the `remove_duplicates_sorted_ll(ll)` Function

```
def remove_duplicates_sorted_ll(ll):  
    current = ll.head  
    if current is None:  
        return  
  
    while current.next is not None:  
        if current.item == current.next.item:  
            temp = current.next.next  
            current.next = temp  
            ll.size -= 1  
        else:  
            current = current.next
```

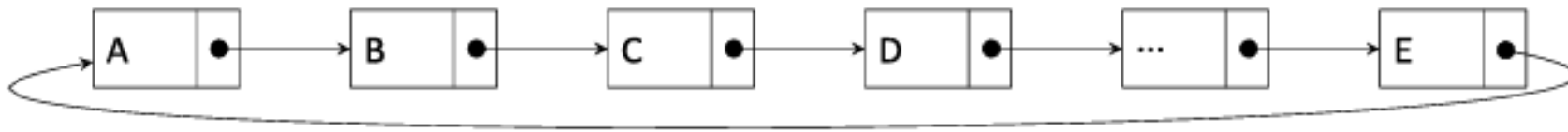
- If values are different, move **current** one step forward.



Q04

We assign the link of the last node to the first node instead of assigning it to a null value. This turns the linked list into a circular linked list. Let Aptr and Bptr point to any two nodes in the linked list. What is the outcome of the following functions?

Figure 1.1: A Circular Linked List



Q04 - Answer

Let's go through the given T2Q1 (s,q) Function

```
def T2Q1(s, q):  
    temp = s  
    while temp.next != q:    # move forward until next is q  
        temp = temp.next  
    temp.next = s            # rewire so that temp.next points to s
```

What does T2Q1(s, q) actually do?

- It **starts at s** and walks forward until it finds the node whose .next is q.
- Once it finds that node (let's call it prev_of_q), it **rewires** prev_of_q.next to point to s instead of q.
- This **breaks** the original circular path and forms a **smaller loop** starting at s.

Applying T2Q1(Aptr, Bptr)

Let's use the example where Aptr points to B and Bptr points to D:

For T2Q1(Aptr, Bptr):

1. Start at B ($s = \text{Aptr} = \text{B}$)
2. Traverse until we find the node before D (which is C)
3. Make C point back to B
4. Result: $\text{B} \rightarrow \text{C} \rightarrow \text{B}$ becomes one circle

For T2Q1(Bptr, Aptr):

1. Start at D ($s = \text{Bptr} = \text{D}$)
2. Traverse until we find the node before B (which is A)
3. Make A point back to D
4. Result: $\text{D} \rightarrow \text{E} \rightarrow \text{A} \rightarrow \text{D}$ becomes another circle

Q04 - Answer

Applying T2Q1(Aptr, Bptr)

- One circular list starts at **Aptr** and includes nodes up to (and including) the node BEFORE **Bptr**. This means **B** $\rightarrow$ **C** $\rightarrow$ **B** forms the first circle.
- Another circular list starts at **Bptr** and includes nodes up to (and including) the node BEFORE **Aptr**. This means **D** $\rightarrow$ **E** $\rightarrow$ **A** $\rightarrow$ **D** forms the second circle.